AI Chatbot with Streamlit + Ollama

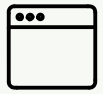
A practical local-AI project that turns a Python interface into a conversational assistant.

Made with GAMMA

THE PROJECT

The project turns a local model into a usable chatbot

Three pieces connect a prompt to a visible response.



Streamlit

Provides the interactive web interface.



Ollama

Runs and serves the language model locally.



Conversation state

Connects user prompts, model responses, and history.

A small application surface, a local model service, and state that keeps the conversation coherent.

Made with GAMMA

SYSTEM MAP

Architecture: three layers, one conversation loop

The user enters a prompt, Python forwards the conversation, and Streamlit renders the model's response.

Ollama's local server uses the base URL `http://localhost:11434/api`; local requests do not need an authorization header.

Made with GAMMA

THE FRONT END

Streamlit keeps the interface simple

The UI has only a few jobs, and each one stays visible.



Capture

Chat input captures the next user message.



Separate

Chat message containers distinguish user and assistant turns.



Preserve

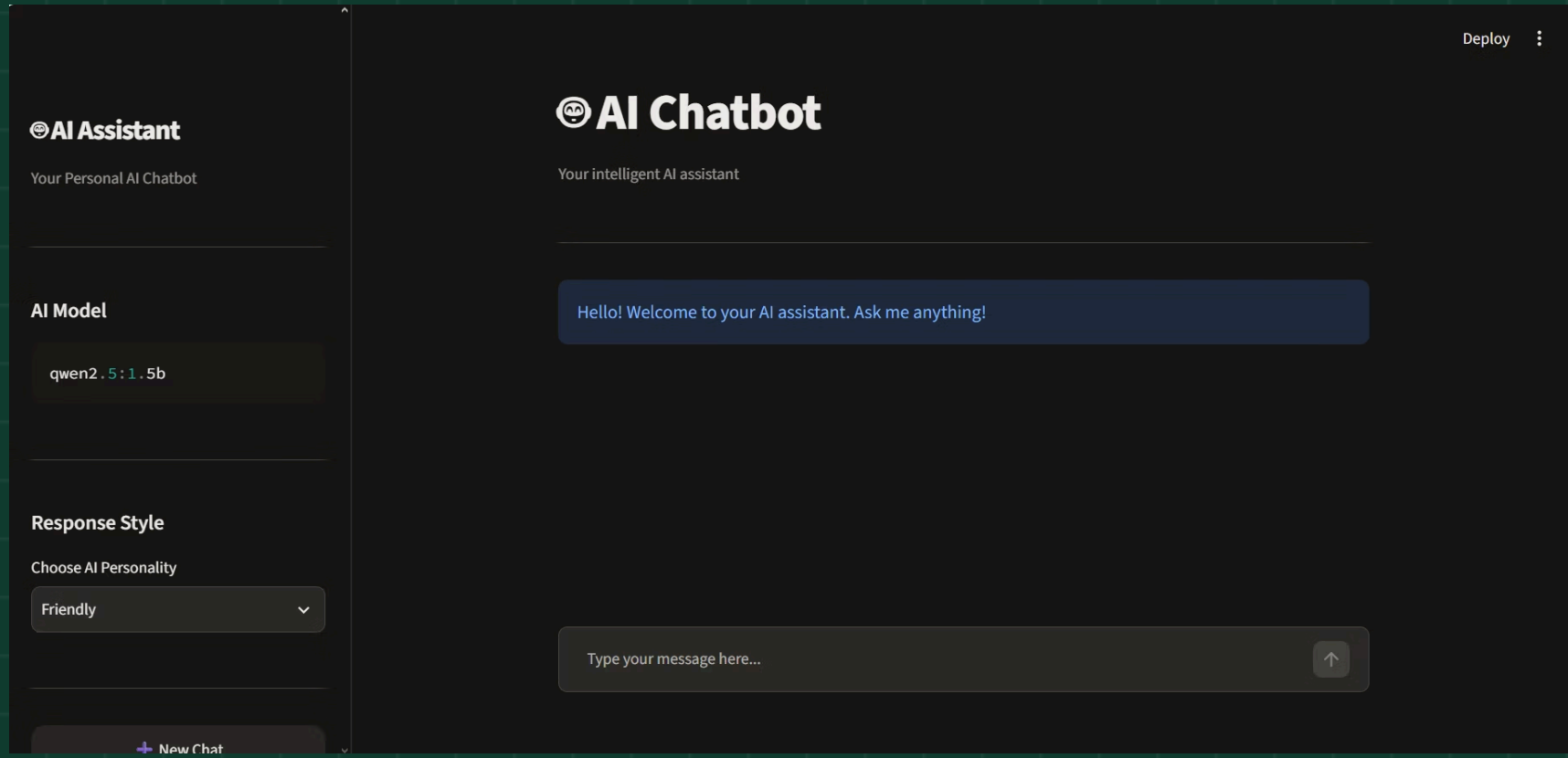
Session state preserves the conversation while the app runs.

Made with GAMMA

INTERFACE OVERVIEW

Dark chatbot interface

Sidebar with AI Assistant branding and model settings, the AI Chatbot heading, a welcome message, and the chat input.

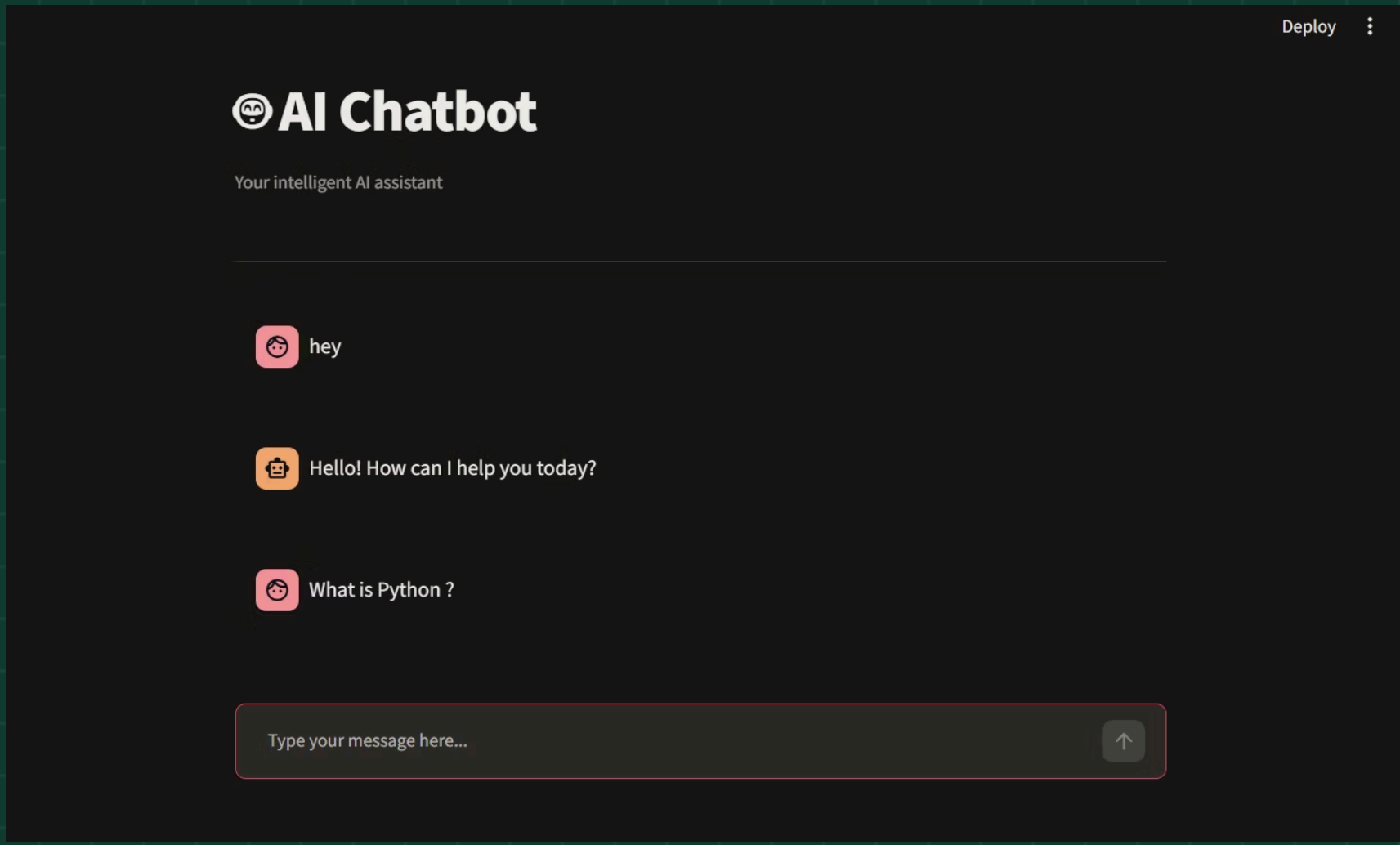


Made with GAMMA

CHAT WINDOW

A short conversation and message input

The dark chat window shows a brief exchange above the message box, ready for the next prompt.

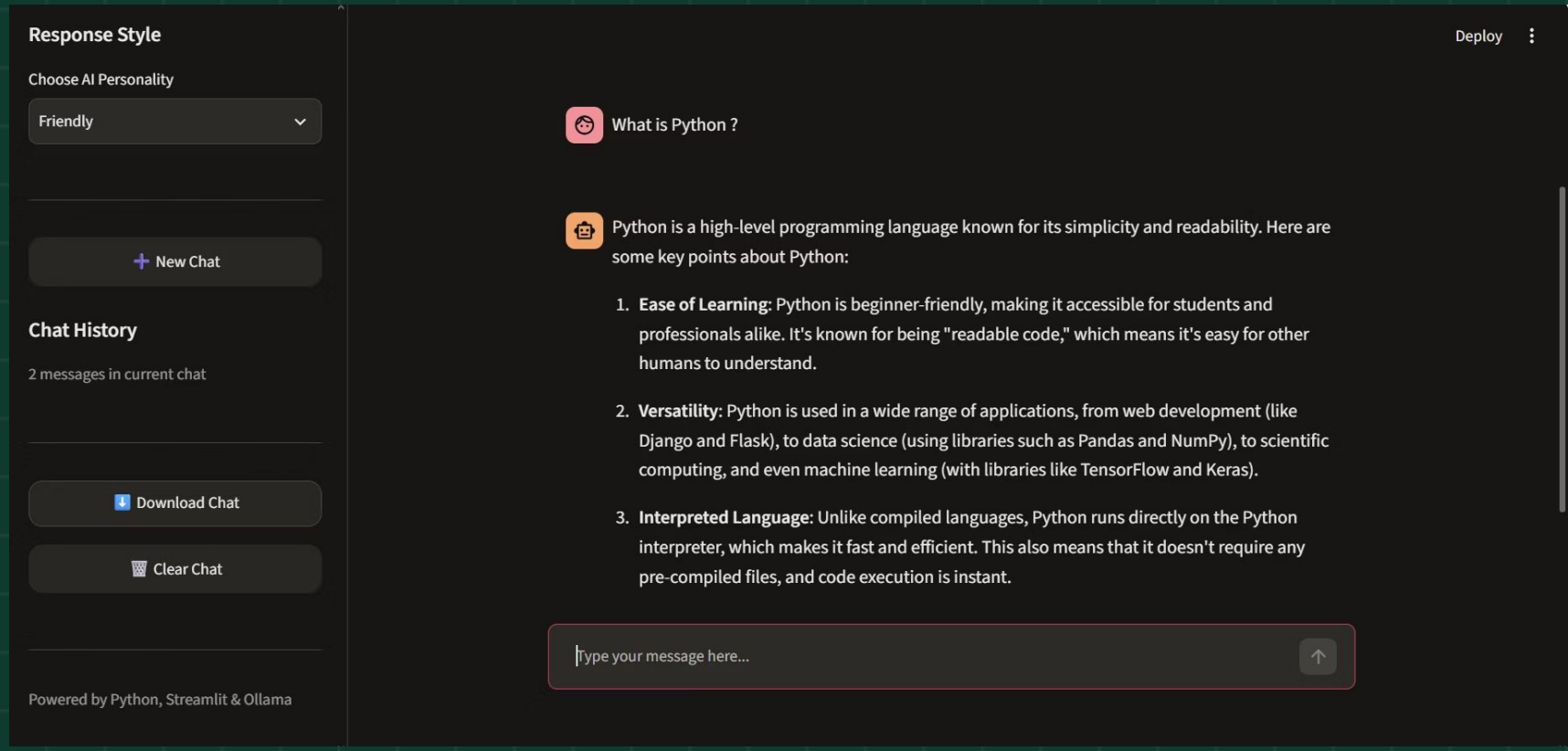


Made with GAMMA

Q&A IN ACTION

A detailed answer in Friendly style

A Python question gets a structured answer, with the Friendly response style and chat controls visible.

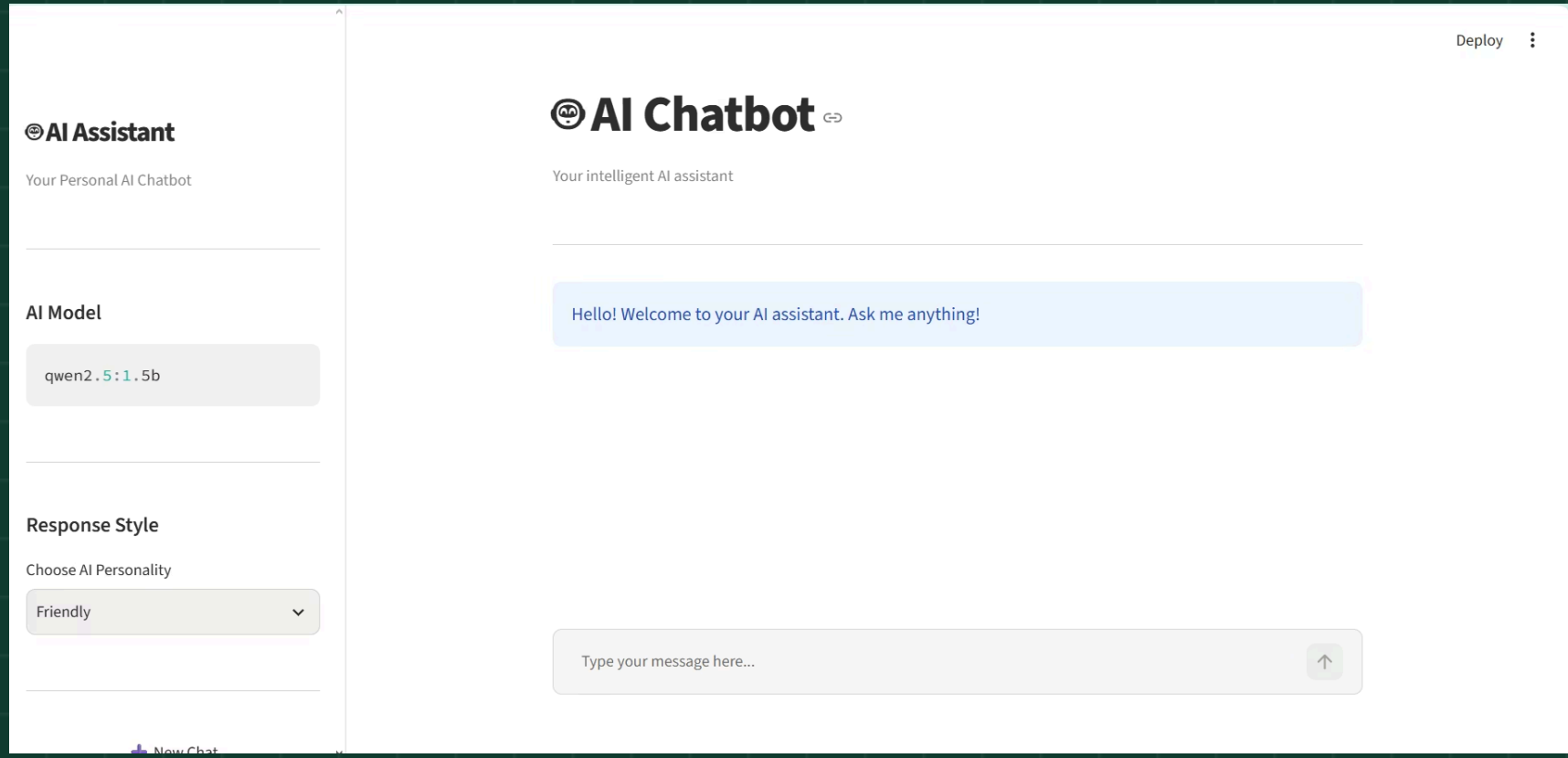


Made with GAMMA

LIGHT THEME

The same layout in light theme

The light theme shows the qwen2.5:1.5b model selector and the Friendly response style setting.

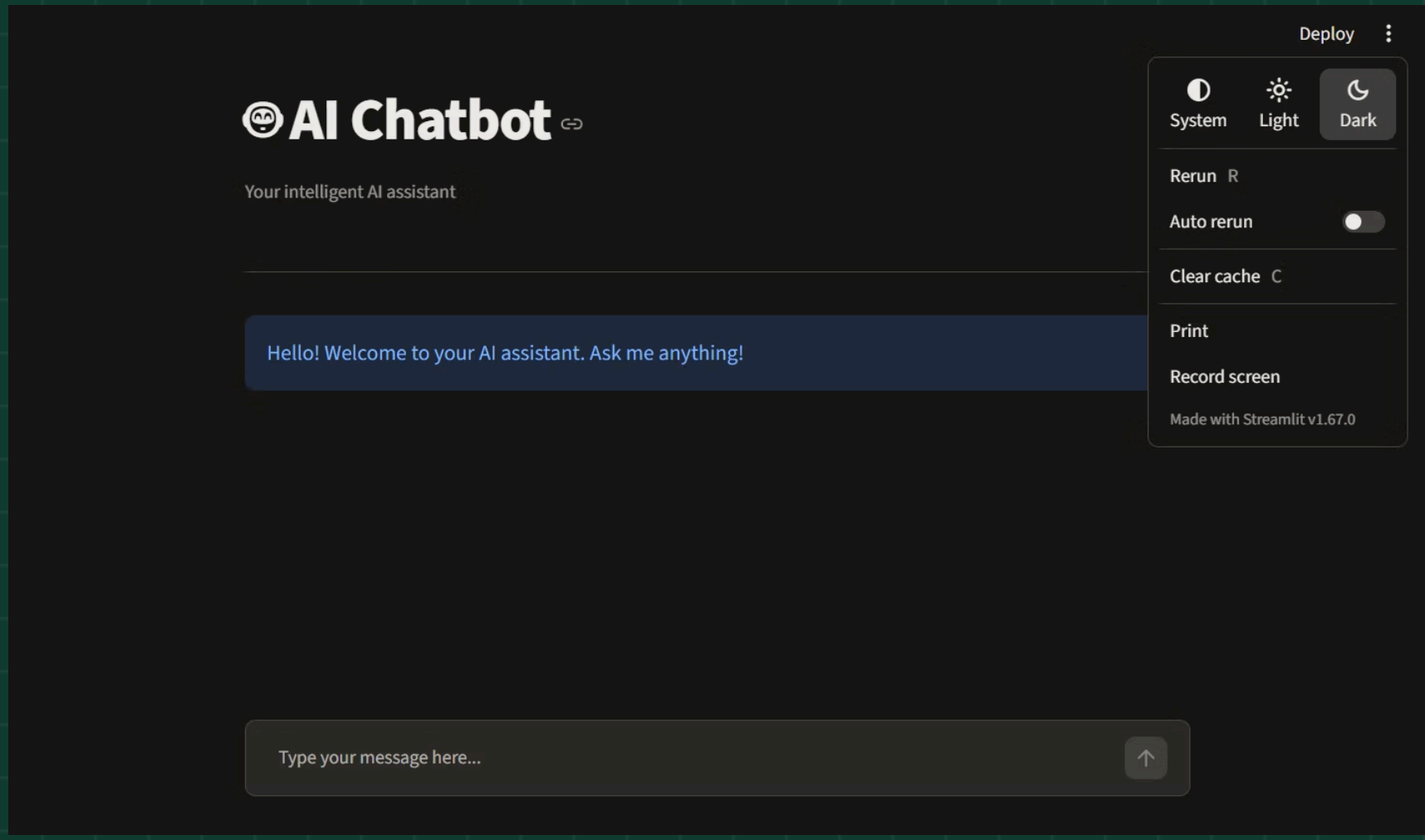


Made with GAMMA

THEME CONTROL

Choose System, Light, or Dark

The theme menu offers System, Light, and Dark, with Dark selected.

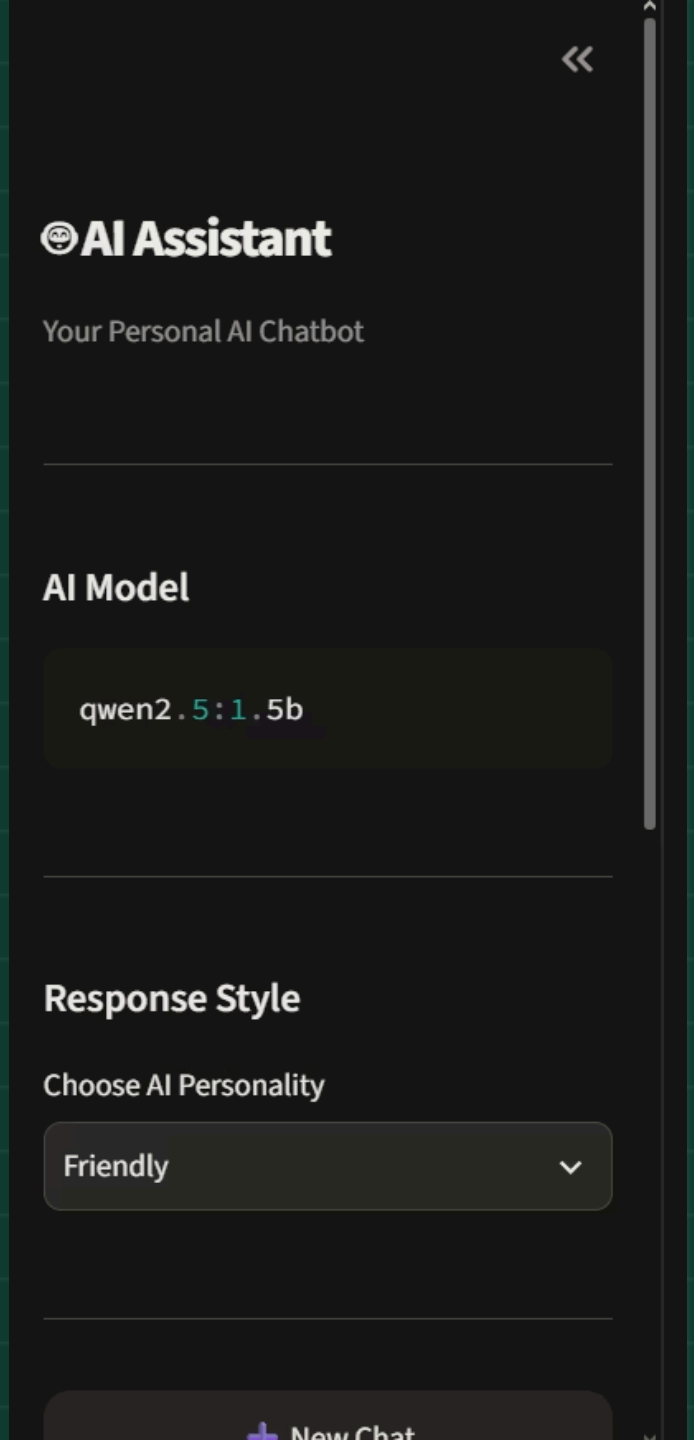


Made with GAMMA

SIDEBAR

Model and response style in the sidebar

The sidebar shows AI Assistant branding, the qwen2.5:1.5b model, and the Friendly personality setting.

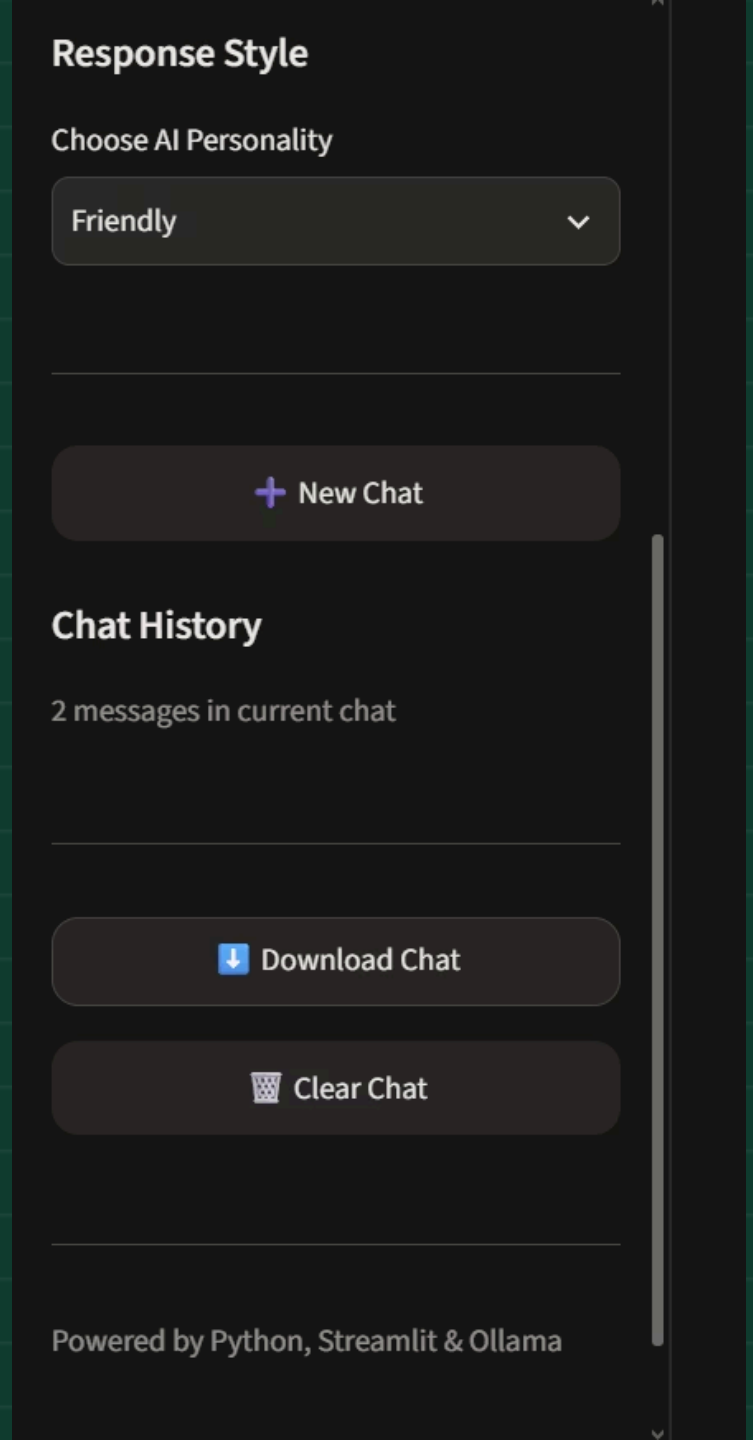


Made with GAMMA

SIDEBAR ACTIONS

Chat actions stay one click away

New Chat, Chat History, Download Chat, and Clear Chat sit above a Python, Streamlit and Ollama footer.

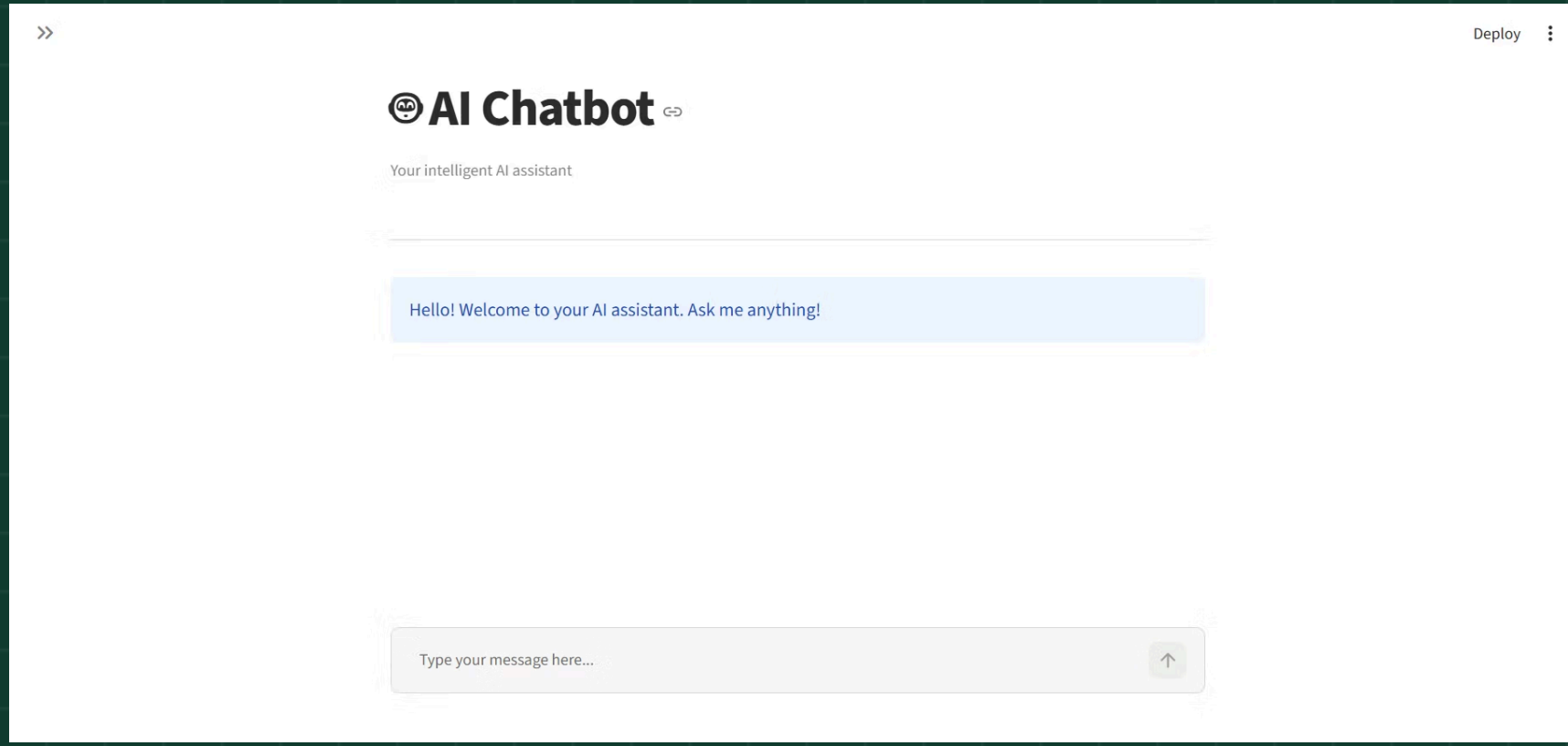


Made with GAMMA

FOCUS MODE

The workspace with the sidebar hidden

With the sidebar hidden, the chat takes the full window, leaving the messages and input in focus.



Made with GAMMA

THE MODEL CONNECTION

Ollama handles the model connection.

It keeps the model boundary straightforward: install, send, read.

01

Install

Install Ollama and download a local model.

02

Send

Send a model name plus the message history.

03

Read

Read the returned assistant content and display it.

LOCAL MODEL RUNTIME

Ollama supports local models and provides official Python and JavaScript libraries.

Made with GAMMA

TRACE THE LOOP

The request cycle is easy to trace

01 Render prior messages

02 Wait for new input

03 Append the user message

04 Call Ollama

05 Append and render the assistant response

Made with GAMMA

IMPLEMENTATION PATTERN

Core implementation pattern

Keep the moving parts easy to replace, test, and explain.

Single chat function

Use a single chat function for the model call.

Separate configuration

Keep configuration separate from the UI.

Handle failure clearly

Explain connection errors and missing models.

Change with confidence

Keep prompts and model selection easy to change.

Made with GAMMA

DEMO FLOW

From prompt to response

The demo makes the architecture observable in a few moves.

1

Start

Start the Ollama service.

2

Launch

Launch the Streamlit app.

3

Ask

Ask a practical question.

4

Show

Show the response and preserved chat history.

Made with GAMMA

PROJECT TAKEAWAY

What the project demonstrates

A compact build that makes the full application path concrete.



Python structure

Organize a working application around clear responsibilities.



Streamlit UI

Build a usable interface without a heavy front-end layer.



Local API integration

Connect the interface to a model through Ollama.



State and errors

Preserve context and make failures understandable.

Made with GAMMA